# MIT 8103 Advanced Database Systems: Portfolio Summary

**Project Title:** Design and Implementation of an Advanced Database System for Digital Banking and Transaction Management
**Student Name:** Crystal Nkemeh
**Student ID:** 301921032
**Institution:** Miva Open University
**Academic Session:** 2026/2027
**GitHub Repository:** https://github.com/Crystall14/Crystal_Nkemeh_301921032

# Case study

PurpleBank is a fictional Nigerian digital bank designed to support customer registration, account management, deposits, withdrawals, internal transfers, and simulated interbank transfers. The project uses fictional records and focuses on maintaining data integrity, efficient transaction processing, and reliable database operations.

PostgreSQL was selected for core relational banking records, while MongoDB was used to demonstrate flexible document-based storage of banking activity events. All financial transactions are denominated in Nigerian Naira (NGN).

# Portfolio 1: Database Design and Modelling

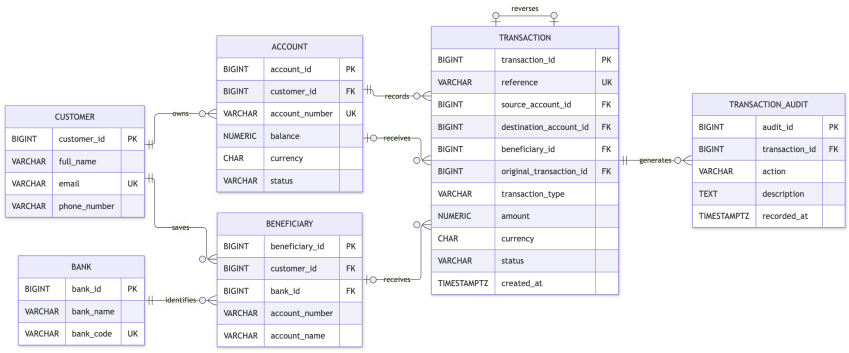
I designed and implemented a relational database containing six entities: Customer, Account, Bank, Beneficiary, Transaction, and Transaction Audit.

An Entity-Relationship diagram was developed to illustrate the relationships between these entities, including one-to-many relationships between customers and their bank accounts. Primary keys, foreign keys, and database constraints were defined to enforce data integrity.

The database structure was normalised to Third Normal Form (3NF) to reduce redundancy and prevent update, insertion, and deletion anomalies.

Using PostgreSQL, I created the six tables and implemented constraints including NOT NULL, UNIQUE, FOREIGN KEY, and CHECK. Fictional records were inserted to test database operations, while deliberately invalid records were used to verify constraint enforcement.

### ER Diagram



# Portfolio 2: Query Processing and Optimisation

Three SQL queries were developed to analyse PurpleBank's banking records. They retrieved customer transfers, identified accounts without outgoing transfers, and calculated transaction counts per customer.

To evaluate query performance, I generated 100,000 fictional transaction-history records in a separate benchmark table.

The experiment compared a date-filtered query before and after creating a B-tree index on the created_at column using EXPLAIN (ANALYZE, BUFFERS).

| Measurement | Before Index | After Index |
|---|---|---|
| Execution plan | Sequential Scan | Index Only Scan |
| Median execution time | 8.652 ms | 0.050 ms |
| Shared buffer hits | 1,152 | 3 |
| Matching records | 24 | 24 |

After indexing, the median execution time decreased by approximately **99.42%**, representing an improvement of about **173 times**.

The results demonstrate how appropriate indexing can reduce database-page access and improve query performance. However, performance improvements depend on the query, data distribution, and execution environment.

# Portfolio 3: Transactions and Concurrency

PostgreSQL transaction management was demonstrated using COMMIT and ROLLBACK.

A successful NGN 10,000 deduction was committed, changing a fictional customer's account balance from NGN 100,000 to NGN 90,000. A subsequent transaction temporarily deducted another NGN 5,000 but encountered a negative-balance constraint violation. ROLLBACK cancelled the unfinished changes and preserved the previously committed balance.

A concurrency experiment was also conducted using two PostgreSQL sessions operating at the READ COMMITTED isolation level.

Session A inserted an uncommitted transaction record. Session B could not retrieve that uncommitted record and encountered a conflicting insertion attempt using the same transaction reference.

Following Session A's COMMIT, the database's unique constraint rejected the duplicate insertion in Session B. The experiment demonstrated transaction isolation, constraint enforcement, and concurrency control.

# Portfolio 4: NoSQL and Advanced Data Models

MongoDB was introduced as a complementary database for storing flexible banking activity records. A database named purplebank_events and a collection named transaction_events were created. The document model incorporated embedded customer information, transaction details, timestamps, and event-specific metadata. Four fictional banking events were designed to represent transfer initiation, transfer completion, withdrawal completion, and a failed interbank transfer. MongoDB queries were used to count event documents, retrieve the history of a specified transfer, and identify failed banking activities. The document model illustrates MongoDB's flexibility in storing varying event attributes, such as failure reasons and processing times, without requiring identical structures across all records.

# Portfolio 5: Distributed and Cloud Database Exercise

A distributed database architecture was proposed for PurpleBank's MongoDB banking activity records. The design consists of a three-member MongoDB replica set, with one primary and two secondary servers deployed across separate failure zones. Replication would maintain copies of activity records, while majority-based elections could support primary failover when sufficient voting members remain available. Majority write acknowledgement was proposed to improve the durability of acknowledged events, although asynchronous replication may cause temporary differences between members. Sharding was also considered as a future scalability option if transaction activity volumes increased significantly. This architecture remains a **design proposal**. No distributed MongoDB cluster, sharding configuration, or cloud deployment was implemented.

# Evidence

The GitHub repository contains the database design documents, ER diagram, normalisation examples, SQL scripts, MongoDB implementation, execution plans, constraint tests, transaction demonstrations, and supporting screenshots. Evidence is organised across five portfolio folders corresponding to the assessment activities. The repository preserves progressive development through Git commits.

# Individual reflection

Although I had prior experience working with SQL and databases, this project gave me a better understanding of what goes into designing and implementing a database system from scratch. I particularly enjoyed the query optimisation exercise because I could see the difference indexing made to execution time, rather than just reading about its benefits.

The concurrency exercise was also insightful, especially seeing how PostgreSQL handled two sessions attempting conflicting operations. I encountered a few challenges with the development environment and database setup, but working through them improved my confidence in troubleshooting technical issues.

Overall, the project helped me connect database concepts to practical scenarios and better appreciate the importance of data integrity, performance, and reliability in banking systems.

# References

ArizonaL2014. (n.d.). *MIT 8103 Advanced Database Systems Codelab*. GitHub. https://github.com/arizonal2014/MIT8103_Codelab

MongoDB. (n.d.). *MongoDB documentation*. https://www.mongodb.com/docs/

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. https://www.postgresql.org/docs/current/

# AI-use declaration

I used OpenAI's ChatGPT to explain database concepts, assist with the development and adaptation of SQL and MongoDB scripts for the PurpleBank case study, structure my portfolio documentation, and interpret some query execution plans and test results.

I worked through the practical exercises in GitHub Codespaces and used the resulting database outputs and screenshots to review the implementation. The final submission will be checked against my saved scripts, actual results, and assessment requirements. I remain responsible for understanding, verifying, and explaining the work submitted.